EE231001 Introduction to Programming
Midterm Project Report: Tomb Explorer

Student ID	[your student ID]	**Name**	[your name]
Course	EE231001, Prof. Andy (Yu-Guang Chen)	**Due**	2026/11/22 23:59:59
Files	`StudID_Name_MP.cpp` (basic, 393 lines)	`StudID_Name_MP_bonus.cpp` (bonus, 607 lines)	

Summary. Both programs are complete C++11 implementations of the project specification. The basic program covers every section of the PDF (seeded dungeon generation with BFS validation, hearts / stamina / backpack rules, monsters, weapons and drops, the recursive DFS for the theoretical maximum value, and the exact input/output format). The bonus program adds the Automatic Pathfinding Mode (`r`) as a breadth-first search over *game states*, so the best route is found without enumerating paths. **The source code was written with an AI coding assistant**; Section 3 explains exactly how.

1. How to compile and run

Environment. GNU g++ with the C++11 standard (the grading workstation setup). The development machine used g++ 13.3.0 on Ubuntu. Only the standard library is used (`iostream, iomanip, string`, plus `vector` in the bonus).

Version	Compile	Run
Basic	`g++ -std=c++11 -Wall -Wextra -o PA_studID.out StudID_Name_MP.cpp`	`./PA_studID.out` or with a prepared input file: `./PA_studID.out < case.in`
Bonus	`g++ -std=c++11 -Wall -Wextra -o PA_studID.out StudID_Name_MP_bonus.cpp`	

Input. The program prompts for the seed, the backpack capacity, then one *weight value* pair for each of the six treasures and the three monster drops; after that it reads one command per prompt (`w a s d`, plus `r` in the bonus) and `y/n` at item prompts. Because tokens are read with `cin >>`, an input file may separate tokens by spaces or newlines. The complete input of the winning game shown below (setup, 40 moves and the four `y/n` answers) is:

```
12345 30 3 50 4 60 2 30 5 80 1 10 6 70 2 40 3 45 1 20 s d s s y w d d d d d d d y a a a s a s y s s y s s s s d d d s s s s s d d d d d d d
```

When input is redirected from a file nothing echoes the typed characters, so the prompts follow each other on one line; in an interactive terminal the typed input appears after each prompt as in the figures below (typed input is shown in **blue**; the captures were made with the real programs, seed 12345, capacity 30, treasure/drop weight-value pairs 3 50 / 4 60 / 2 30 / 5 80 / 1 10 / 6 70 / 2 40 / 3 45 / 1 20).

seed 12345, capacity 30 (setup, first display)

```
=============
Tomb Explorer
=============
Seed: 12345
Backpack capacity: 30
Treasure 1 weight and value: 3 50
Treasure 2 weight and value: 4 60
Treasure 3 weight and value: 2 30
Treasure 4 weight and value: 5 80
Treasure 5 weight and value: 1 10
Treasure 6 weight and value: 6 70
Monster Z drop weight and value: 2 40
Monster V drop weight and value: 3 45
Monster W drop weight and value: 1 20
##################
#U....###....#.###
#....#.#w#....#..#
##........6...#.##
##2.....#.#..##..#
#..###3##..#.#..##
##..##.#.....###.#
#....#5####.###.#.#
#.##.#v#.#.#.#.#.#
#1...#.#.#.#WZ.V##
#.###...####..####
######.......#.#.#
#...#####..###...#
##.#...#...##....#
#####.##...####.##
#...##.##..###z..#
##.#.#.........E#
##################
========== Status ==========
Hearts: 3 / 3
Stamina: 45 / 45
Weapons: z[N] v[N] w[N]
Total Value: 0
Backpack Weight: 0 / 30
Steps: 0
============================
Command [w/a/s/d]:
```

invalid command, border, wall

```
Command [w/a/s/d]: x
Invalid command.
Command [w/a/s/d]: w
You cannot move outside the map.
Command [w/a/s/d]: d
   ... 3 more uneventful moves (d d d) ...
Command [w/a/s/d]: d
You hit a wall.
Command [w/a/s/d]:
```

treasure prompt (invalid answer, then y)

```
   ... 7 more uneventful moves (s s d d d d d) ...
Command [w/a/s/d]: d
You found Treasure 6.
Weight: 6, Value: 70
Take it [y/n]: q
Please enter y or n.
Take it [y/n]: y
Item collected.
Command [w/a/s/d]:
```

monster without its weapon

```
   ... 10 more uneventful moves (s s d d d d d d w d) ...
Command [w/a/s/d]: w
Monster Z blocked you.
You need weapon z and lost 1 heart.
Command [w/a/s/d]:
```

Figure 1. Left: the initialisation output, the first map (18 x 18 with the wall border; `U` = player, `S` start, `E` exit, digits = treasures, `Z V W` monsters, `z v w` weapons), the status block and the first prompt. Right: invalid command, border and wall bumps (free), a treasure prompt with an invalid answer, and a monster attack without the weapon (the heart and stamina cost can be read from the status block that follows). After every command the 26-line map + status display is printed again; it is omitted in the excerpts to save space.

a complete winning game (compacted)

```
Command [w/a/s/d]: s
   ... 2 more uneventful moves (d s) ...
Command [w/a/s/d]: s
You found Treasure 2.
Weight: 4, Value: 60
Take it [y/n]: y
Item collected.
Command [w/a/s/d]: w
   ... 7 more uneventful moves (d d d d d d d) ...
Command [w/a/s/d]: d
You found Treasure 6.
Weight: 6, Value: 70
Take it [y/n]: y
Item collected.
Command [w/a/s/d]: a
   ... 4 more uneventful moves (a a s a) ...
Command [w/a/s/d]: s
You found Treasure 3.
Weight: 2, Value: 30
Take it [y/n]: y
Item collected.
Command [w/a/s/d]: s
Command [w/a/s/d]: s
You found Treasure 5.
Weight: 1, Value: 10
Take it [y/n]: y
Item collected.
Command [w/a/s/d]: s
You picked up weapon v.
Command [w/a/s/d]: s
   ... 16 more uneventful moves (s s d d d s s s s s d d d d d d) ...
Command [w/a/s/d]: d
==================================
Tomb Explorer Result
==================================
You escaped the tomb.
Final Value: 170
Theoretical Max Value: 405
Mission Completeness: 42.0%
Backpack Weight: 13 / 30
Hearts: 3 / 3
Stamina: 5 / 45
Weapons: z[N] v[Y] w[N]
Total Steps: 40
==================================
```

final status and result block

```
========== Status ==========
Hearts: 3 / 3
Stamina: 5 / 45
Weapons: z[N] v[Y] w[N]
Total Value: 170
Backpack Weight: 13 / 30
Steps: 40
============================
==================================
Tomb Explorer Result
==================================
You escaped the tomb.
Final Value: 170
Theoretical Max Value: 405
Mission Completeness: 42.0%
Backpack Weight: 13 / 30
Hearts: 3 / 3
Stamina: 5 / 45
Weapons: z[N] v[Y] w[N]
Total Steps: 40
==================================
```

Figure 2. Left: a complete winning game (the 40-move route computed by the bonus planner, typed by hand into the *basic* program; runs of uneventful moves are merged). Right: the final map, status and result block printed when the exit is reached. Final value 170 of a theoretical maximum 405 (42.0%).

2. Completion of the assignment

Every requirement of the PDF is implemented. The table maps each specification section to the code (function names refer to the source files).

Specification	Implementation	Status
2 Random number generator	`seedRng()`, `getRandomNum()`: state = seed mod 2^{31}; next = (state · 1103515245 + 12345) mod 2^{31}; returns `state >> 16`. 64-bit arithmetic, so the product cannot overflow.	**Done**
3 Map generation, steps 1-2	`generateWalls()`: 256 cells left-to-right, top-to-bottom, each a wall with 40% probability; then `S` at (0,0) and `E` at (15,15) are forced.	**Done (decision D1)**
Steps 3-4 BFS, shortest path	`runBfs()`: 4-direction BFS fills `reachable[][]` and `dist[][]` and returns D. `buildDungeon()` rejects the dungeon and draws the next one from the same random stream until the exit is reachable.	**Done**
Step 5 Object placement	`placeObject()` in the order 1-6, Z, V, W, z, v, w; row = rand % 16, then col = rand % 16; retried until the cell is reachable and still `.`.	**Done**
4 Movement	`handleMove()`: w/a/s/d only; border and wall bumps print a message and cost nothing; any other token is *Invalid command.*	**Done**
5-7 Hearts, monsters, weapons, drops	Monster without its weapon: -1 heart, -1 stamina, +1 step, monster and player stay. With the weapon: *You defeated monster X.*, the player moves in and the cell becomes the drop (`a/b/c`). Weapons are picked up automatically, are permanent and weigh nothing.	**Done**
8-9 Treasures, backpack	`offerItem()`: prints name, weight, value, asks *Take it [y/n]:* (upper case accepted), repeats on invalid answers, checks total weight ≤ capacity; declined or too-heavy items stay on the map.	**Done**
10 Stamina	Max stamina = ceil(1.5 D) computed as `(3D+1)/2`; each successful move or monster-blocked attempt costs 1; wall/border bumps are free.	**Done**
11 Theoretical maximum value	`searchBestValue()`: **recursive DFS** over take/skip for the 9 items, pruned by capacity and by an optimistic bound (value so far + all remaining positive values cannot beat the best).	**Done**
12 Mission completeness	collected / maximum × 100 with exactly one decimal digit; 0.0% when the maximum is 0.	**Done**
13-14 Win / lose	Entering `E` ends the game as a win; hearts = 0 or stamina = 0 ends it as a loss, with the specified messages and final result block.	**Done**
Input / output format	All prompts, messages, the status block and the result block follow the PDF text.	**Done**
Bonus	Automatic Pathfinding Mode in `StudID_Name_MP_bonus.cpp` (Section 5).	**Done**

Restrictions

Restriction	How it is met
At least one recursive function	`searchBestValue()` calls itself for the "take" and the "skip" branch.
No `struct` / `class`	None is declared (checked with a search of both files). A map cell is the integer `row*16+col`; the BFS queues and the bonus search records are plain arrays / `vector<int>`; the game state lives in global variables.
No brute force	Reachability is a BFS; the knapsack DFS prunes; the bonus planner searches the space of *states* (about 10^4-10^5 in practice) instead of the space of *paths* (up to $4^{45} \approx 10^{27}$ walks within the stamina limit).
g++, C++11, naming	Compiled with `-std=c++11`. The files are named `StudID_Name_MP.cpp` / `StudID_Name_MP_bonus.cpp` in the repository and must be renamed with the real ID and name for submission.

Testing and verification

No reference implementation or sample expected output was available, so correctness was checked by *differential testing*: independent Python re-implementations of the game rules and of the bonus planner (a different algorithm: layered sets, backward marking of optimal states, forward greedy walk) were fed the same random input streams as the C++ programs and the full transcripts were compared character by character.

Check	Result
Compiler warnings	0 warnings with `-std=c++11 -Wall -Wextra -Wpedantic -Wshadow -Wconversion` (both files).
Basic program vs. oracle	450 random games (invalid commands, border and wall bumps, monster hits, invalid/upper-case answers, negative and > 2^{31} seeds, capacities 0-39): **0 mismatches**. Endings: 141 escapes, 228 hearts, 81 stamina.
Bonus program vs. oracle	1,660 random games, the planner was triggered in 1,305 of them (at the start or after manual play with item pick-ups, weapon pick-ups and lost hearts); 216 of those found no route: **0 mismatches** in route, value, weight, per-step events and final result.
Memory/UB checks	All fuzz runs used AddressSanitizer + UndefinedBehaviorSanitizer builds: no reports.
Scenario coverage (399 auto runs from the start)	166 chains weapon → monster → drop, 273 collected items, 154 "left it here", 250 weapon pick-ups, 64 "no route".
Auto vs. manual consistency	The planner's 40-move route (seed 12345) typed manually into the basic program ends with exactly the predicted value 170, weight 13 / 30 and 40 steps (Figure 2).
Speed / memory of the planner	about 10 ms and 13 MB peak even with an oversized backpack (capacity 1000).

> **Limits of this evidence.** The oracles were written from the same reading of the PDF as the programs, so they catch coding mistakes but not a different interpretation of an ambiguous sentence. The decisions below are those interpretations. [Student: after running the course's public test cases, write the result here. They were not available when this draft was prepared.]

Interpretation decisions (where the PDF is silent)

#	Question	Choice (and where to change it)
D1	How does the random number become "40% wall"?	`getRandomNum() % 100 < 40` (constant `WALL_PERCENT` in `generateWalls()`). The common alternative is `% 10 < 4`; it changes every dungeon.
D2	Prompt layout	Prompts end with a space and no newline. The map and status are printed before *every* prompt, including after *Invalid command.*, wall and border bumps. After an invalid y/n answer the prompt *Take it [y/n]:* is repeated.
D3	Order of end-of-move checks	Cell events (weapon, drop, treasure prompt) first; then exit reached = win (even with 0 stamina left); then stamina = 0 = loss. Hearts = 0 ends the game before the stamina check.
D4	Letter case	Movement commands are lower case only; `y/n` accept `Y/N`; multi-character tokens are invalid.
D5	Negative seeds	Normalised into [0, 2^{31}) so that `% 16` can never be negative.
D6	Bonus tie between plans that differ only in take/leave	"Take" is explored before "leave". No blank line is printed between *Route:* and *Auto Step 1*.
D7	End of input	If stdin ends mid-game the program exits quietly instead of looping.

3. How AI was used

> **Declaration.** The program source of both files, the README and the first draft of this report were produced with an AI coding assistant (**Claude Code**, by Anthropic), working from the project PDF. The AI also wrote and ran the test tools (Python oracles and fuzzers, not part of the submission). The same disclosure is written at the top of both source files.

What the AI did, step by step

1. **Read the specification** (all 24 pages, including figures, output samples, the bonus chapter and the restrictions) and listed the points where the text is ambiguous (Section 2, D1-D7).
2. **Chose the architecture**: global arrays instead of structs; BFS for reachability; a recursive DFS for the theoretical maximum; for the bonus a BFS over packed game states instead of path enumeration.
3. **Wrote the basic program** (`StudID_Name_MP.cpp`), compiled it with strict warnings and smoke-tested it.
4. **Built a verification harness**: an independent Python implementation of the game and an adaptive fuzzer that drives the compiled program with a semi-intelligent bot and compares transcripts. The harness first had bugs of its own (a walker that kept typing movement keys into a y/n prompt;

off-by-one coordinates while preparing the report captures); these were test-tool bugs, not program bugs.

5. **Wrote the bonus** by extending the basic file, then a second oracle with a different algorithm and ran the 1,660-game comparison. The only change made to the program after that was renaming a local variable to remove a `-Wshadow` warning; the tests were re-run afterwards.
6. **Drafted the README and this report**; all transcripts in the report are captured from the real programs.

Which parts of the code were AI-assisted

Part of the code	AI involvement	Reviewed / changed / verified by the student
RNG, `generateWalls()`, `runBfs()`, `placeObject()`, `buildDungeon()`	written by AI	[Student: for each group of functions, state whether you read it line by line, whether you changed anything, and how you checked it yourself.]
`searchBestValue()`, `computeTheoreticalMax()` (recursion)	written by AI	
Output: `printMap()`, `printStatus()`, `printResult()`	written by AI	
`handleMove()`, `offerItem()`, `main()` loop	written by AI	
Bonus: `planRoute()`, `pushState()`, `packState()`, `runAutoMode()`	written by AI	
Test tools, README, report draft	written by AI	

Student's own account (required by the course). [Student: describe in your own words how you used the AI and what you personally did. Cover: what you asked it, which parts you read and understood, anything you modified or re-wrote, what you compiled and tested yourself, and what you could and could not explain at the demo. Be specific and truthful; the course treats undisclosed AI help as plagiarism.]

4. Difficulties and how they were overcome

4.1 A specification with silent spots

Several details that decide the exact output are not in the PDF: how a random number becomes a 40% wall probability, whether prompts end in a newline, what is redisplayed after a rejected command, and what happens when stamina and hearts reach zero in the same move. Because the same seed must give the same dungeon, a wrong guess about the first one changes *every* map. **Approach:** each choice was made explicit (decisions D1-D7), isolated in one constant or one function so that it can be changed in one place, and documented.

4.2 The bonus: the number of paths explodes

The automatic mode must collect the maximum value within the remaining stamina. A search over *routes* has up to 4 choices per step and the stamina allows about 45 steps, i.e. up to $4^{45} \approx 1.2\times10^{27}$ walks. That is brute force and is prohibited. **Approach:** search *states* instead. A state (Figure 3) is the player's cell, the weapons held, the monsters defeated and the items taken (23 bits). Everything else follows from it: weight and value from the items, steps used from the BFS depth, remaining stamina from the depth. The future of a state does not depend on how it was reached, so each state needs to be expanded only once. With visited flags the search touched a median of about 2×10^4 states (maximum 3.3×10^5 in 480 random instances), against 8,388,608 possible states, and takes about 10 ms.

Backpack capacity	median states explored	95th percentile	maximum
10	10,025	48,938	78,523
30	23,319	109,526	332,383
100	19,663	107,984	179,638
1000	20,004	103,482	210,456

120 random dungeons per capacity (random weights 1-7 and values 0-99), start-of-game search, measured with the Python oracle, which explores the same state set as the C++ planner.

4.3 Guaranteeing "fewest steps, then first found in the order s, w, d, a"

The rule is easy to state but easy to get subtly wrong. **Idea:** a BFS with "visited on discovery" reaches every state by its alphabetically smallest shortest route. By induction on the depth: entries of one depth are queued in the alphabetical order of their routes; children are generated parent by parent and direction by direction in the order `s, w, d, a`, so the next depth is queued in the same order, and the first discovery of a state is its smallest route. Moreover, on a shortest optimal route every state lies at its minimum depth (otherwise a shorter prefix would give a shorter route to the same goal state), so remembering one parent per state loses no optimal route. The goal state is therefore chosen by "highest value; then smallest depth; then earliest discovered" (strict comparisons while the BFS runs). This was confirmed against an oracle that finds the alphabetically smallest optimal route with a completely different method.

4.4 No `struct` or `class`

A search state naturally wants a record. **Approach:** pack it into one `int` (`packState()`), keep the BFS queue as parallel `vector`s (state, parent, depth, move, took-item flag), and encode a map cell as `row*16+col`.

4.5 Verifying without a reference answer

With no sample output to compare to, the risk is bugs that are invisible on a few hand-played games. **Approach:** differential testing against independent implementations on thousands of random games (Section 2), plus a cross-check that mixes the two programs: the route found by the bonus is replayed by hand in the basic program and must reproduce the predicted value, weight and step count.

5. Bonus: Automatic Pathfinding Mode

Implemented in `StudID_Name_MP_bonus.cpp`. Typing `r` at the prompt (now `Command [w/a/s/d/r]:`) plans and executes a route from the *current* game state.

Bonus requirement	Implementation
Search order s, w, d, a	`PATH_CMD[] = {'s','w','d','a'}`; the BFS expands the directions in that order.
Reach the exit within remaining stamina	Expansion stops at depth = current stamina; `E` is terminal (entering it ends the expedition).
Maximum collected value within the remaining capacity	Every treasure / drop is a branch: take (only if `weight + item ≤ capacity`) or leave; the goal state with the highest total value wins.
Weapons picked up; monsters only entered with the weapon	Weapons are picked up on contact (forced transition); a monster cell is entered only if the weapon is held, which also defeats the monster and exposes its drop.
Same value → fewest steps → first found	BFS depth, then discovery order (Section 4.3).
Uses the current state (position, stamina, weight, value, items, weapons, defeated monsters)	The start state is built from the live game variables; `origGrid` keeps the generated layout so that the planner does not depend on what was already picked up.
Automatic item selection	The plan records one take/leave flag per step; during execution *Item collected.* or *Auto mode left it here.* is printed without any y/n prompt.
Output format	*Auto Pathfinding Mode*, *Best reachable value / Expected backpack weight / Route length / Route*, *Auto Step X / L: C* before each move, map and status after each move, *No route to the exit was found from the current state.*

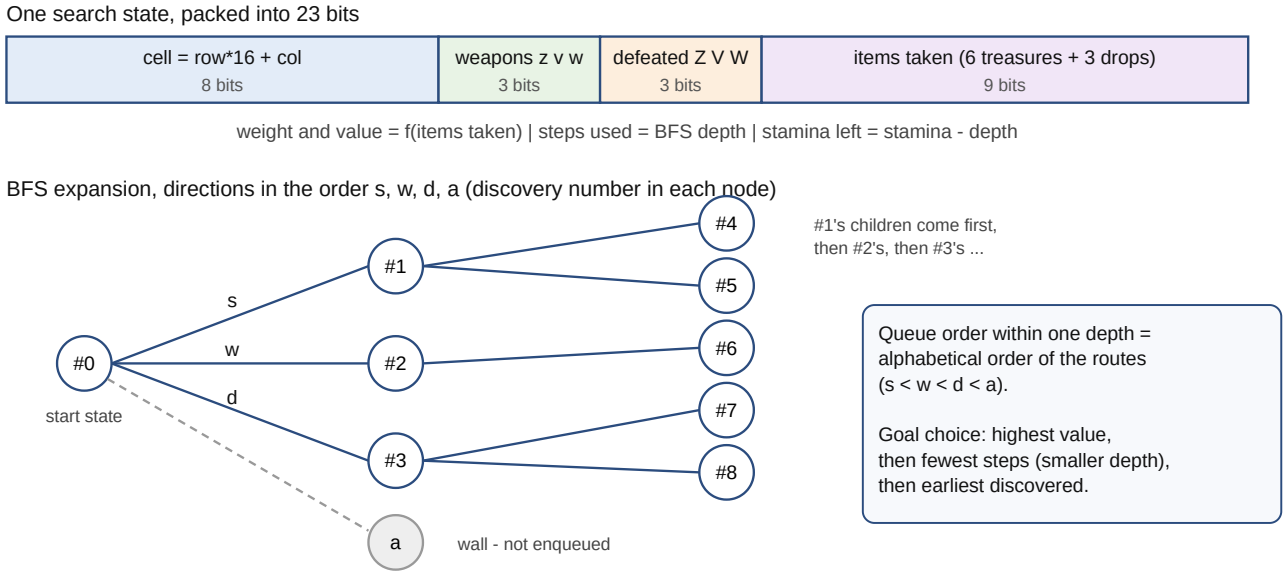


Figure 3. The planner's state and the BFS discovery order. Each node is one state; the numbers are the order in which the BFS discovers them, which equals the alphabetical order of the routes that reach them.

Pseudocode of the planner

```
planRoute():
    start = pack(playerCell, weaponsHeld, monstersDefeated, itemsTaken);  queue = [start];  seen[start] = true
    best = none
    for each entry e of the queue, in order:
        if cell(e) is the exit  or  depth(e) == stamina:  continue
        for dir in (s, w, d, a):
            cell' = neighbour of cell(e) in dir;  skip if outside the map or a wall
            if cell' holds a monster:   skip unless its weapon is held;  mark it defeated;  item = its drop
            if cell' holds a weapon:    mark the weapon as held
            if cell' holds a treasure:  item = that treasure
            options = [ leave ]
            if item exists and is not yet taken and weight(items) + weight(item) <= capacity:
                options = [ take, leave ]                # "take" is tried first
            for option in options:
                s' = pack(cell', weapons', defeated', items' )
                if not seen[s']:  seen[s'] = true;  enqueue(s', parent = e, move = dir, tookItem = option == take)
                    if cell' is the exit and  (value(items') > bestValue  or  equal value and depth+1 < bestDepth):
                        best = this entry
    if best is none: return "no route"
    follow the parent links from best back to the start  ->  route and take/leave flags
```

Sample runs (real output)

'r' at the start (seed 12345, capacity 30)

```
Command [w/a/s/d/r]: r
Auto Pathfinding Mode
Best reachable value: 170
Expected backpack weight: 13 / 30
Route length: 40
Route: sdsswddddddddaaasasssssssdddsssssddddddd
Auto Step 1 / 40: s
   ... 2 more uneventful moves (d s) ...
Auto Step 4 / 40: s
You found Treasure 2.
Weight: 4, Value: 60
Item collected.
Auto Step 5 / 40: w
   ... 7 more uneventful moves (d d d d d d d) ...
Auto Step 13 / 40: d
You found Treasure 6.
Weight: 6, Value: 70
Item collected.
Auto Step 14 / 40: a
   ... 4 more uneventful moves (a a s a) ...
Auto Step 19 / 40: s
You found Treasure 3.
Weight: 2, Value: 30
Item collected.
Auto Step 20 / 40: s
Auto Step 21 / 40: s
You found Treasure 5.
Weight: 1, Value: 10
Item collected.
Auto Step 22 / 40: s
You picked up weapon v.
Auto Step 23 / 40: s
   ... 16 more uneventful moves (s s d d d s s s s s d d d d d d) ...
Auto Step 40 / 40: d
====================================
Tomb Explorer Result
====================================
You escaped the tomb.
Final Value: 170
Theoretical Max Value: 405
Mission Completeness: 42.0%
Backpack Weight: 13 / 30
Hearts: 3 / 3
Stamina: 5 / 45
Weapons: z[N] v[Y] w[N]
Total Steps: 40
====================================
```

'r' pressed after taking Treasure 6 by hand

```
Command [w/a/s/d/r]: s
   ... 9 more uneventful moves (d s d d d d d d d) ...
Command [w/a/s/d/r]: d
You found Treasure 6.
Weight: 6, Value: 70
Take it [y/n]: y
Item collected.
Command [w/a/s/d/r]: r
Auto Pathfinding Mode
Best reachable value: 110
Expected backpack weight: 9 / 30
Route length: 27
Route: aaasasssssssddsssssddddddd
Auto Step 1 / 27: a
   ... 4 more uneventful moves (a a s a) ...
Auto Step 6 / 27: s
You found Treasure 3.
Weight: 2, Value: 30
Item collected.
Auto Step 7 / 27: s
Auto Step 8 / 27: s
You found Treasure 5.
Weight: 1, Value: 10
Item collected.
Auto Step 9 / 27: s
You picked up weapon v.
Auto Step 10 / 27: s
   ... 16 more uneventful moves (s s d d d s s s s s d d d d d d) ...
Auto Step 27 / 27: d
====================================
Tomb Explorer Result
====================================
You escaped the tomb.
Final Value: 110
Theoretical Max Value: 405
Mission Completeness: 27.2%
Backpack Weight: 9 / 30
Hearts: 3 / 3
Stamina: 7 / 45
Weapons: z[N] v[Y] w[N]
Total Steps: 38
====================================
```

Figure 4. Left: `r` at the start, seed 12345. The planner picks Treasures 2, 6, 3 and 5 (value 170, weight 13 / 30) with a 40-step route; no prompts appear, items on the route are simply collected. The map and status that follow every step are omitted. Right: after taking Treasure 6 by hand, `r` plans from the *current* state: less stamina is left, so the best reachable value is now 110 with a 27-step route.

seed 1, capacity 30: weapon -> monster -> drop

```
Command [w/a/s/d/r]: r
Auto Pathfinding Mode
Best reachable value: 320
Expected backpack weight: 21 / 30
Route length: 54
Route: sddddddsssssssdsdddwdwwddddsssaddssassaaaaassdawwddsdsdsdd
   ... steps 1-12 omitted ...
Auto Step 13 / 54: d
You picked up weapon z.
Auto Step 14 / 54: s
   ... 8 more uneventful moves (d d d w d w w d) ...
Auto Step 23 / 54: d
You defeated monster Z.
You found Monster Z drop.
Weight: 2, Value: 40
Item collected.
   ... steps 24-53 omitted ...
Auto Step 54 / 54: d
====================================
Tomb Explorer Result
====================================
You escaped the tomb.
Final Value: 320
Theoretical Max Value: 405
Mission Completeness: 79.0%
Backpack Weight: 21 / 30
Hearts: 3 / 3
Stamina: 0 / 54
Weapons: z[Y] v[Y] w[Y]
Total Steps: 54
====================================
```

seed 8: no route

```
Command [w/a/s/d/r]: r
Auto Pathfinding Mode
No route to the exit was found from the current state.
Command [w/a/s/d/r]:
```

Seed 8: the exit cannot be reached without crossing a monster whose weapon is not available, so the mode reports that no route exists and returns to manual play (the map, status and the prompt are printed again).

an item the planner leaves behind

```
capacity 8, seed 1:
Best reachable value: 140
Expected backpack weight: 8 / 8
   ...
Auto Step 5 / 44: d
You found Treasure 1.
Weight: 3, Value: 50
Auto mode left it here.
```

With a backpack of capacity 8 the best selection is worth 140 and fills it exactly; the planner walks over Treasure 1 (weight 3, value 50) and leaves it, because taking it would make a better combination impossible.

Figure 5. Left: a run where the best route picks up a weapon, defeats a monster with it and collects the monster's drop automatically (seed 1, capacity 30). Right: the no-route case, and an item that is left on purpose.

6. Thoughts and suggestions

What the project teaches well

- Deterministic pseudo-randomness: reproducing a dungeon from a seed forces precise reading of an algorithm, because one different `%` changes everything.
- Two classic searches used for what they are good at: BFS for shortest distance and reachability, DFS for exhaustive-but-pruned selection (knapsack).
- In the bonus, the move from "enumerate paths" to "search states" is a genuine modelling step: choosing what the state must contain, and proving that nothing else is needed.

Suggestions for the specification

1. State how the random number becomes the 40% wall probability (for example `% 100 < 40`). This single sentence decides whether every map is right.
2. Provide one complete worked example: a seed with the resulting map, D, maximum stamina and the first screens, so students can check their generator before building the rest.
3. Say whether prompts end with a newline or a space, and whether the map and status are redisplayed after an invalid command or a wall bump.
4. Define the order of the end-of-move checks (item prompt on the last stamina point, stamina and hearts reaching 0 together).
5. Clarify "brute force": Section 11 asks for a DFS over all 2^9 combinations, which is itself exhaustive; the wording could say which exhaustive searches are allowed.
6. Bonus samples contain blank lines between the printed lines; saying whether those are real output (or only the document's spacing) would remove a guess.

> **Student's own thoughts.** [Student: your own reflections on the assignment: what was hard for you, what you learned, what you would do differently.]

Appendix A. Control flow and pseudocode of the basic program

```
buildDungeon():
    repeat
        for r in 0..15, c in 0..15:  grid[r][c] = (getRandomNum() % 100 < 40) ? '#' : '.'
        grid[0][0] = 'S';  grid[15][15] = 'E'
        D = bfs(0,0)                          # fills reachable[][] ; returns dist to E or -1
    until D >= 0                              # the random stream is NOT reset between attempts
    for symbol in "123456ZVWzvw":
        repeat  row = getRandomNum() % 16;  col = getRandomNum() % 16
        until reachable[row][col] and grid[row][col] == '.'
        grid[row][col] = symbol
    maxStamina = ceil(1.5 * D)

searchBestValue(i, weight, value):           # recursive DFS, items 0..8
    best = max(best, value)
    if i == 9 or value + (sum of positive values of items i..8) <= best:  return
    if weight + w[i] <= capacity:  searchBestValue(i+1, weight + w[i], value + v[i])    # take
    searchBestValue(i+1, weight, value)                                                 # skip

handleMove(cmd):
    (nr,nc) = player + direction(cmd)
    if outside the map: print "You cannot move outside the map." ; return        # free
    if wall:            print "You hit a wall." ; return                         # free
    stamina--;  steps++
    if monster at (nr,nc) and its weapon is missing:
        hearts--;  print "Monster X blocked you." ...;  if hearts == 0 -> lose;  elif stamina == 0 -> lose;  return
    move the player
    if monster: print "You defeated monster X.";  cell becomes its drop
    if weapon:  pick it up;  cell becomes '.'
    if treasure / drop:  offerItem()         # ask y/n until valid; capacity check
    if cell is the exit: win   elif stamina == 0: lose

main():
    read seed, capacity, 9 weight/value pairs;  buildDungeon();  theoreticalMax = DFS
    while game running:  print map + status;  read command;  w/a/s/d -> handleMove ; (r -> auto mode) ; else "Invalid command."
    print map + status + result block
```

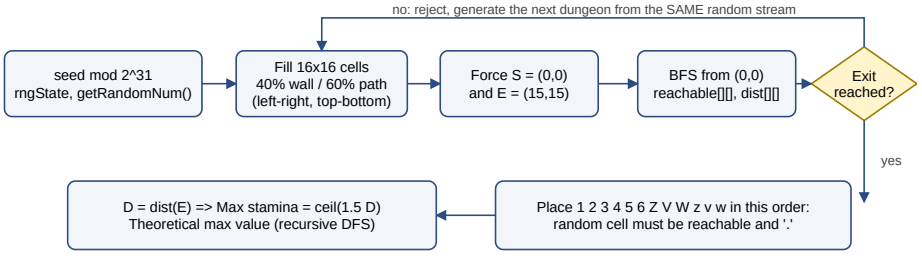


Figure A1. Dungeon generation (Section 3 of the PDF) as implemented in `buildDungeon()`.

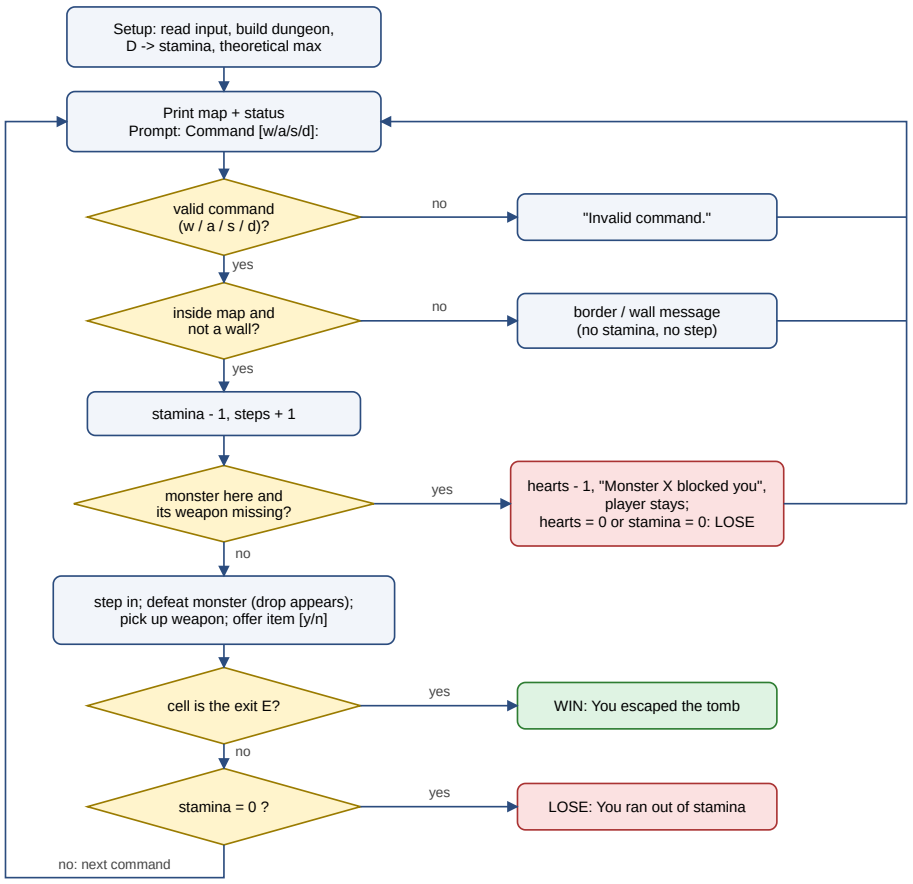


Figure A2. Control flow of one command in the game loop (`main()` and `handleMove()`).

Appendix B. Function index

Function	Purpose
`seedRng, getRandomNum`	pseudo-random generator from the PDF
`generateWalls, runBfs, placeObject, buildDungeon`	dungeon generation, validation and object placement
`searchBestValue, computeTheoreticalMax`	recursive DFS for the theoretical maximum value
`printMap, printStatus, printWeapons, printResult`	all formatted output blocks
`offerItem, collectItem*`	item prompt, capacity check, collection (*bonus file only)
`handleMove`	one movement command: costs, monsters, weapons, drops, win/lose checks
`readSetup, main`	input of the setup values and the game loop
`fillMaskTables, packState, pushState, planRoute, runAutoMode*`	bonus: item-subset tables, state packing, BFS queue, planner, route execution (*bonus file only)